

DAG tokenizer using greedy sem cov

yuanyuan.zheng

August 2026

1 Introduction

1.1 Background

- *EHR Foundation Model (FM) arises nowadays, with potential to be assistive clinical decision tool*
- *they often use standardized clinical vocabularies such as ICD, RXNORM, SNOMED, LOINC*
- *standardized clinical vocabularies /ontologies DAG, are used to normalize clinical naming, and improve the interoperability*
- *SNOMED CT is a DAG, comprehensive 400k+ concepts (covering very broad clinical domains, disorders, medication, morphologies etc...) and semantically rich (composed by 100+ relationship types, have their own way to compose machine readable semantic expression), mappable to other clinical terminologies.*

Electronic health record (EHR) foundation models (FMs) are a recent class of models that learn from large amounts of patient history, with the goal of eventually assisting clinical decisions (e.g. Li et al., 2020, BEHRT; Yang et al., 2023, TransformEHR). To train on data from many hospitals at once, these models cannot rely on free text alone, because the same clinical fact is written differently in different places. Instead, they are trained on standardized clinical vocabularies – ICD, RxNorm, SNOMED CT, LOINC – so that the same fact always gets the same code **[CITE: standardized clinical vocabulary / interoperability reference]**.

Several of these vocabularies are more than flat lists of codes: they are ontologies, organized as directed acyclic graphs (DAGs), in which a concept is connected to other concepts that define it. SNOMED CT is the largest and most detailed example. It contains more than 400,000 concepts, covering disorders, procedures, medications, and more. Its concepts are connected by over 100 relationship types, so that each concept can be described as a machine-readable combination of others, and it can be mapped onto most other major clinical terminologies **[CITE: SNOMED International concept model / release statistics]**. **[DOUBT: worth cross-checking against Su et al. (2025, MedTok)’s own count – they report 303,325 SNOMED CT concepts]**

specifically (out of 617,490 codes total across 8 systems: ICD-10-CM 81,184, RxNorm 81,151, ICD-10-PCS 61,644, NDC 54,560, ICD-9 18,365, CPT 10,602, ATC 6,659) – likely a different SNOMED edition/extension than your 400k figure; worth a one-line reconciliation rather than leaving two different SNOMED counts unexplained in the same paper.]

1.2 Gap

- *training FM needs either large amount of patient data (but patient data is difficult to get)*
- *or need adequate size of clinical vocabularies. (but medical vocabularies are large size)*
- *Current methods are:*
- *(cite) LLM byte-pair on categorical medical concept: data-driven, no semantic preservation, highly dependant to data quality and concept orders.*
- *(cite) keep highest prevalent top k concepts, highest entropy contribution top k concepts: data-driven, loss of considerable clinical information.*
- *(cite) truncate/reduce leaf concepts to their parents concepts: data-driven, loss of medical granularity.*
- *(cite) or just keep all concepts as it is, if we have lots of patient data. but since concept usage distribution are usually long tail, many rare concepts can not be learnt in a proper way.*
- *(cite) learnable embedding space codebook: loss of controllability and interpretability*
- *all of above mentioned methods are ignoring the semantic richness of clinical vocabulary DAG when try to deal with vocabulary size.*

This scale creates a problem for training an EHR FM. Learning a good representation for every one of these concepts takes a large amount of patient data, and such data is expensive and often hard to access **[DOUBT: do you want to cite/quantify this, e.g. typical cohort sizes needed vs. available, or leave it as a general statement?]**. Reducing the vocabulary to a size the available data can actually support does not avoid the problem either: concept usage in real EHR data follows a long tail, so most concepts are already too rare to learn well, no matter how the vocabulary is chosen. Su et al. (2025, MedTok) report the same pattern independently – “low-frequency codes suffer from poor embeddings, reducing performance on underrepresented conditions” – though, like us, without an actual frequency plot to back it up. **[DOUBT: would strengthen this sentence a lot to show your own long-tail plot from the mapped-concept usage data, if you have it, rather than relying on two qualitative claims (yours and MedTok’s) with no numbers behind either.]**

Several strategies already exist for shrinking the vocabulary, and Table 1 summarizes them side by side. Byte-pair and other subword encodings merge concepts by co-occurrence statistics alone (Sennrich et al., 2016; Kudo & Richardson, 2018, SentencePiece): they need no ontology, but the resulting vocabulary depends heavily on data quality and on the arbitrary order in which concepts appear. Keeping only the most frequent, or highest-information, concepts (top- k pruning) drops a significant amount of clinically relevant information simply because it is rare [CITE: prevalence/entropy-based vocabulary pruning in clinical ML]. Rolling leaf concepts up to their ontology parent shrinks the vocabulary but loses clinical detail: a specific diagnosis collapses into a broader one [CITE: ontology roll-up / ICD truncation approach]. Keeping the full vocabulary avoids all of these losses, but only postpones the long-tail problem to the model, and only works if enough data is available [DOUBT: is there a citation you have in mind for this "no reduction" baseline, or is it meant as the trivial comparison point rather than a cited method?]. Finally, learnable embedding-space codebooks compress the vocabulary without any hand-written rules (Van den Oord et al., 2017), but the resulting tokens are then just indices into a learned space: they no longer correspond to concepts a person can recognize or check.

Approach	Uses the ontology graph?	Tokens stay inspectable?	Main limitation
Byte-pair / sub-word encoding	No	No (merges are statistical, not semantic)	Sensitive to data quality and concept order
Frequency / entropy top- k	No	Yes (kept items are still real concepts)	Drops clinically relevant but rare concepts
Hierarchy truncation / roll-up	Partially (parent-child links only)	Yes, but coarser	Loses clinical granularity
Full vocabulary (no reduction)	N/A	Yes	Long tail is never solved, just handed to the model
Learnable code-book (e.g. Med-Tok)	Yes (e.g. via a graph encoder)	No (opaque codebook indices)	Not human-auditable
Ours	Yes, directly	Yes, fully traceable	(this paper’s contribution)

Table 1: How existing vocabulary-reduction strategies compare on the two properties this paper cares about most: whether they use the ontology’s graph structure, and whether the resulting tokens can be inspected by a person.

Table 1 makes the gap concrete: no existing approach combines the two

properties we want – using the DAG’s structure *and* producing tokens a person can inspect. [DOUBT: Is the sharpest way to state the gap ”none of these use the DAG structure itself,” or the stronger claim ”none of these produce a token set that is both semantically motivated *and* inspectable/controllable”? The second framing lines up more directly with the contribution list below and with the table – your call on which to lead with.]

Closest related work. One method in the table needs a closer look, because it is the exception to ”no graph structure”: Su et al. (2025, MedTok) feed each code’s neighborhood in a biomedical knowledge graph, along with its text description, into a multimodal encoder, then compress the result into a shared, learned codebook. So the real difference between our method and MedTok is not ”uses the graph” versus ”does not” – it is *how*. MedTok’s tokens are indices into a continuous space learned by a neural network: there is no way to look at a token and say which relationships in the graph produced it. Our tokens, by contrast, are chosen directly from the graph by a deterministic rule (Section 3), and every token a concept resolves to comes with an explicit record of which relationships were followed to reach it. [DOUBT: this paragraph is my synthesis of the distinction, not something either paper states outright – worth double-checking it reads as fair to MedTok (it’s a real, well-evaluated method, not a strawman) rather than as dismissive.]

1.3 Our method and contribution

- *applicable to DAG knowledge graph.*
- *tokens selection: Represent concepts by smaller vocabulary size using greedy token selection approach*
- *way to tokenize: Represent concepts by a tree/subgraph preserving the structure of the concepts the semantic value.*
- *By apply to SNOMED CT concepts used in our local institut real data, we show our method can control how we tokenize and with what we tokenize (controllability and interpretability)*
- *(maybe) if any mapping inter-ontology exist, we can use concepts in 1 ontology to represent/tokenize concepts in the other ontologies (interoperability)*

Building on this gap, we propose a method for selecting and building tokens directly on a concept DAG such as SNOMED CT. It has two parts. First, we choose a small set of tokens with a greedy algorithm that maximizes how well they cover the meaning of the target concepts (Section 3). This lets us fix the vocabulary size in advance, instead of it falling out of a training run. Second, instead of representing a concept as an unordered set of the tokens it maps to, we represent it as a *context tree*: a small subgraph that records which relationship led to each token. This matters in practice – for example, a token for ”kidney” and a token for ”left” stay attached to the same anatomical-site branch, instead of being mixed up with an unrelated ”right adrenal gland”

branch found elsewhere in the same concept **[DOUBT: this kidney/adrenal example is carried over from your existing PDF draft – keep it, or swap in a real example from the HUG-mapped concepts once we have one handy?]**.

We apply this method to the SNOMED CT concepts mapped by our institution’s semantician team. The result, unlike every approach in Table 1 including MedTok, is directly controllable and inspectable: for any concept, a domain expert can see exactly which relationships were used to reach which token.

[NOTE: Per your feedback: the inter-ontology point (last bullet above) is not being claimed as a contribution of this paper – it’s mentioned only in Future Work (Section 7) now, not developed here.]

2 Background

2.1 Directed acyclic graphs

- *DAG (directed acyclic graph)*

A directed acyclic graph (DAG) is a directed graph $G = (V, E)$ with no directed cycle: there is no path that starts and ends at the same vertex while always following edge direction. Equivalently, G admits a topological ordering of V in which every edge points from an earlier to a later vertex. Acyclicity is what makes “ancestor” and “descendant” well defined for any two connected vertices: a vertex reached by following edges forward is strictly more specific than, and can never be an ancestor of, the vertex it was reached from. Section 3 relies on this property directly – it is what guarantees every hop-distance used later in the paper is finite and well defined.

2.2 SNOMED CT

- *Number of concepts, number of distinct relations,*
- *DAG nature*
- *semantic value from relation types and concept connectivities (concepts are defined by the others in the DAG)*

SNOMED CT is one such graph. Table 2 summarizes its size and the working numbers used throughout the rest of the paper.

Two properties of this graph matter for everything that follows. First, it is acyclic (Section 2.1 above). Second, its edges carry semantic value beyond hierarchy alone. One relationship type, **IS_A**, encodes generalization/specialization – the parent/child relation defined above. Every other relationship type (for example *finding site*, *associated morphology*, *has active ingredient*) attaches an attribute to a concept by pointing to *another concept*, rather than by storing a value locally. As a consequence, a concept’s meaning is not contained in the concept itself: it is constructed from its position and connections inside the DAG. Two concepts that look unrelated by name can share most of their meaning if

Quantity	Value
Total SNOMED CT concepts, $ V $	$\approx 400,000$ [CITE: SNOMED International release statistics]
Distinct relationship types	[RESULTS NEEDED: exact count for the current release/extension]
Mapped concepts (institution-specific), $ M $	$\approx 40,000$
Candidate pool at $D = 9$, $ V_D $	75,964
Weakly connected components of V_D	2

Table 2: SNOMED CT and working-subgraph statistics used throughout the paper. D , M , and V_D are defined in Section 3.1.

they are reached through the same relationships to the same targets; conversely, two occurrences of the same relationship type on the same concept can point to entirely different, non-interchangeable aspects of it. This distinction – **ISA** versus everything else – is exactly what our method is built around: Section 3 defines a coverage score and a tree representation that treat the two kinds of edge differently for this reason.

In practice, we do not draw candidate tokens from all of V : we restrict them to a bounded neighborhood of the mapped concepts M (Section 3.1 gives the exact construction and its depth budget D). Under the current setting ($D = 9$), this candidate subgraph V_D contains **75,964** concepts, spread across **2** weakly connected components (Table 2). *[NOTE: worth a short remark in the paper: this is barely larger than the candidate-pool size computed under the old $D = 3$ setting ($\sim 75,634$), despite D growing from 3 to 9 – the D -hop ego-graph around M saturates fast, and most of the reachable neighborhood was already captured at $D = 3$. Also worth flagging: 2 weakly connected components means the candidate graph is not a single connected structure – does that split correspond to something meaningful (e.g. two disjoint clinical domains in M), or is it a couple of stray/isolated concepts? Good to know before Section 4 describes the pool.]*

2.3 Working vocabulary and notation

With these properties established, we can now state the notation used for the rest of the paper. Let $M \subseteq V$ denote the mapped concepts introduced above, and let $T \subseteq V$, $|T| = k$, denote a *token set*: a small subset of concepts chosen to represent every concept in M . Section 3 defines exactly how T is chosen and how a concept in M is represented once T is fixed.

3 Method

3.1 Candidate pool and depth budget

A single depth budget D plays three roles in this method, all controlled by one hyperparameter, `TokenizerParam.max_dist_candidate` in the implementation: it bounds how far a candidate token may be from the mapped concepts it can help represent, it bounds how deep a context tree (Section 3.3) may grow, and it bounds the horizon over which the coverage score below is computed. Concretely, the candidate pool is

$$V_D = M \cup \{u \in V : u \text{ is reachable from some } c \in M \text{ by at most } D \text{ relationships}\},$$

obtained in practice as the union of the D -hop ego-graph (following outgoing edges) around every mapped concept – see Table 2 for its current size. Token selection (Section 3.2) only ever considers $t \in V_D$: a candidate that no mapped concept can reach within D hops has zero possible contribution to the objective below and is never evaluated.

The current value, $D = 9$, is not an arbitrary round number: it covers 98.4% of concept depths observed in the candidate subgraph, while staying well below the longest path length in the graph (30, driven by a handful of outlier chains). Concepts only reachable beyond depth 9 tend to be generic SNOMED "grouper" categories (e.g. *Wound*, *Inspection (procedure)*) rather than clinically specific concepts, so little is lost by not extending the budget further. **[DOUBT: Using one shared D for all three roles (candidate radius, tree depth, coverage horizon) is a simplifying design choice worth calling out explicitly to a reviewer – is that coupling intentional/load-bearing, or a current implementation convenience that a future version might relax into three independent hyperparameters?]**

3.2 Greedy token selection

- *Greedy token selection based on semantic coverage with lambda as distance penalty (add illustration)*

Given the candidate pool V_D , we now need a rule for choosing which k of its members become tokens. For $u \in V$, $T \subseteq V$, and horizon $h \in \{0, \dots, D\}$, let $S_h(u, T) \in [0, 1]$ denote how well u is covered by T when at most h further relationships may be traversed. The base case is

$$S_0(u, T) = \mathbf{1}[u \in T],$$

and, extending the recursion with a per-hop distance penalty $\lambda \in (0, 1]$ *[NOTE: confirmed by you: this λ term is your own addition on top of the original PDF draft, which only has the undiscounted ($\lambda=1$) version of Eq. (1) – well spotted*

catching that gap.],

$$S_h(u, T) = \begin{cases} 1, & u \in T, \\ 0, & u \notin T \text{ and } d^+(u) = 0, \\ \frac{\lambda}{d^+(u)} \sum_{(u,r,v) \in \text{Out}(u)} S_{h-1}(v, T), & \text{otherwise.} \end{cases} \quad (1)$$

In words: a token is absorbing (its coverage is 1 at every remaining horizon), and every other node’s coverage is the λ -discounted average coverage of its out-neighbors. Setting $\lambda = 1$ recovers the undiscounted score, in which a token found j hops away and a token found at the root contribute equally; for $\lambda < 1$, a token found j hops away instead contributes λ^j , so the score also rewards *proximity* to a token, not only reachability. The overall objective is the mean coverage of the mapped concepts at the full horizon D :

$$F_D(T) = \frac{1}{|M|} \sum_{c \in M} S_D(c, T), \quad F_D(\emptyset) = 0, \quad 0 \leq F_D(T) \leq 1.$$

This objective has a property that makes it tractable at scale: F_D is normalized, monotone, and submodular in T (the marginal gain of adding a token never increases as more tokens are already selected), which follows from writing F_D as a positive average, over random depth- D walks from each $c \in M$, of the indicator that the walk hits T . Submodularity buys two things. First, the classical greedy approximation guarantee,

$$F_D(T_k^{\text{greedy}}) \geq \left(1 - \frac{1}{e}\right) \max_{|T| \leq k} F_D(T). \quad (2)$$

Second, and more important for a candidate pool of tens of thousands of nodes, it licenses *lazy* greedy: each candidate’s previously computed marginal gain is a valid upper bound on its current gain, so at every step it suffices to re-verify only the best stale candidate against the next-best stale bound, instead of recomputing every remaining candidate. Algorithm 1 states this loop precisely; it calls CANDIDATEDELTA (Algorithm 2), which computes one candidate’s exact marginal gain by propagating a sparse *delta* backward through the incoming-edge structure – adding a candidate t only ever changes S_h at t itself and at vertices that can reach t within D hops in reverse – instead of recomputing the full $O(D(|V| + |E|))$ recursion of Eq. (1) from scratch for every candidate at every step.

3.2.1 Precomputation and reuse across a sweep

Two pieces of Algorithms 1–2 depend only on the graph G , never on T or on which candidate is being scored: the row-normalized transition matrix A that Eq. (1) multiplies against (with λ already folded in), and the per-node out-edge lists used by the tokenizer (Section 3.3), pre-sorted so that non-ISA

Algorithm 1 SELECTTOKENS – lazy-greedy token selection

Require: candidate pool V_D , mapped concepts M , horizon D , budget k , penalty λ

Ensure: token set T with $|T| = k$ (or fewer if candidates run out)

```
1:  $T \leftarrow \emptyset$ ;  $S \leftarrow \mathbf{0}$   $\triangleright S_h(u, T)=0$  for all  $u, h$  when  $T = \emptyset$ 
2: for all  $t \in V_D$  do  $\triangleright$  initial seeding pass – exact gains, parallelizable
3:    $(\text{gain}, \_) \leftarrow \text{CANDIDATEDELTA}(t)$ 
4:   push  $(t, \text{gain})$  onto max-heap  $H$ 
5: end for
6: while  $|T| < k$  and  $H \neq \emptyset$  do
7:   repeat
8:      $t \leftarrow \text{pop top of } H$ 
9:      $(\text{gain}, \delta) \leftarrow \text{CANDIDATEDELTA}(t)$   $\triangleright$  re-verify exactly, not from a stale bound
10:     $b \leftarrow$  gain bound now at the top of  $H$  (or  $-\infty$  if empty)
11:    until  $\text{gain} \geq b$   $\triangleright$  submodularity: a stale bound never overstates a candidate's true gain
12:     $T \leftarrow T \cup \{t\}$ ; COMMIT( $t, \delta$ )
13:  end while
14: return  $T$ 
```

relationships are visited before **IS_A** ones. Because a full evaluation sweep reruns selection for every (λ, k) pair on the *same* graph, both structures are built exactly once per graph and shared for the whole sweep – via a process-pool initializer when the initial seeding pass of Algorithm 1 is parallelized – rather than being rebuilt inside every task. This removes a redundant $O(|V| + |E|)$ rebuild from what would otherwise be millions of repeated calls. Only the seeding pass is parallelized this way: once the lazy loop itself is running, an accepted token typically needs only 1–7 re-evaluations of the heap’s top candidate before it is certified as the true maximizer, so the loop is kept serial – at that scale, the overhead of shipping work to a worker process would exceed the work saved.

3.3 Tokenizer

- *tokenizer (say why flatten is not good, add illustration, resulting context trees)*

Selecting T answers “which concepts become tokens.” A separate question is how a mapped concept $c \in M$ should be *represented* once T is fixed. The simplest option – the flat, unordered set of tokens reachable from c – throws away exactly the information that non-**IS_A** relationships were designed to carry: if a concept has two independent *finding site* occurrences, each qualified by its own *laterality*, a flat token set {kidney, adrenal gland, left, right} can no longer tell which laterality belongs to which site. **[DOUBT: keeping this exact kidney/adrenal example from your PDF – confirm, or swap for a real**

Algorithm 2 CANDIDATEDELTA and COMMIT – sparse incremental update

```
1: function CANDIDATEDELTA( $t$ )
2:    $\delta_0 \leftarrow \{t \mapsto 1 - S_0(t)\}$   $\triangleright$  only  $t$  changes at horizon 0
3:   for  $h = 1, \dots, D$  do
4:      $\delta_h \leftarrow \{\}$ 
5:     for all  $(v, d_v) \in \delta_{h-1}$  do
6:       for all  $u \in \text{In}(v)$  with  $u \notin T \cup \{t\}$  do
7:          $\delta_h[u] \leftarrow \delta_h[u] + \lambda d_v / d^+(u)$   $\triangleright$  Eq. (1), propagated backward
8:       end for
9:     end for
10:     $\delta_h[t] \leftarrow 1 - S_h(t)$   $\triangleright$   $t$  is absorbing at every horizon
11:  end for
12:   $\text{gain} \leftarrow \frac{1}{|M|} \sum_{c \in M} \delta_D[c]$ 
13:  return ( $\text{gain}, \delta$ )
14: end function
15: function COMMIT( $t, \delta$ )
16:   for  $h = 0, \dots, D$  do
17:     for all  $(u, d_u) \in \delta_h$  do
18:        $S_h(u) \leftarrow S_h(u) + d_u$ 
19:     end for
20:   end for
21:    $T \leftarrow T \cup \{t\}$ 
22: end function
```

HUG-mapped concept once we pick an illustrative one for the figure below.]

We avoid this by recursively expanding each mapped concept c into a *context tree* instead. **IS_A** relationships are transparent and continue the search in the current context; every other relationship type opens a fresh subcontext, so that two independent occurrences of the same non-**IS_A** relationship type are kept apart. Algorithm 3 states this precisely; two implementation details are worth noting alongside it. First, a node’s outgoing edges are visited non-**IS_A** first, so that when the same subcontext key (r, v) is reached twice at different depths, the shallower (more direct) occurrence is what gets kept (line 9’s “keep more direct” rule in the underlying implementation). Second, a token located exactly at depth D is still accepted, since the token check (line 2) happens before the depth-budget check (line 3).

The resulting tree keeps only T as its vocabulary; its subcontext structure records exactly which relationship chain each token was reached through, which is what preserves the kidney/left vs. adrenal gland/right distinction above.

3.4 Evaluation metrics

- *metrics used and how they are computed,*
- *3.1: Efficiency: conciseness, tree_complexity*

Algorithm 3 EXPAND – context-tree construction, called as EXPAND($c, q_0, 0$)
for each $c \in M$

```

1: function EXPAND( $u, q, d$ )
2:   if  $u \in T$  then
3:     attach  $u$  as a token of context  $q$  at depth  $d$ ; return
4:   else if  $d = D$  or  $u$  has no outgoing edges then
5:     mark  $q$  uncovered on this branch; return
6:   else
7:     for all outgoing edges  $(u, r, v)$  of  $u$  do
8:       if  $r = \text{IS\_A}$  then
9:         EXPAND( $v, q, d + 1$ ) ▷ transparent: stay in the same context
10:      else
11:         $q' \leftarrow$  subcontext of  $q$  keyed by  $(r, v)$ , creating it if new
12:        if  $(r, v)$  was new for  $q$  then
13:          EXPAND( $v, q', d + 1$ )
14:        end if
15:      end if
16:    end for
17:  end if
18: end function

```

[FIGURE PLACEHOLDER – side-by-side: flat token set vs. context tree for a real mapped concept with ≥ 2 non-IS_A relationship occurrences, e.g. the kidney/adrenal-gland example, or a concept pulled directly from the HUG-mapped data.]

Figure 1: Context-tree tokenization example (to be replaced with an actual figure).

- 3.2: *distance_score: Fidelity*
- 3.3: *discriminative power: uniqueness_entropy, unique_rate*
- 3.4: *semantic content: semantic_coverage, uncovered_rate, unk_branch_rate*
- *and also say either they are hogher or lower better (interpretation)*

With a fixed T and context trees for all of M in hand, we can now ask how good this representation actually is. All eight metrics below are computed from the *realized* context trees, as opposed to $F_D(T)$, which is only the objective used to *select* T in the first place (Section 3.2). Table 3 groups them into four families and states, for each, which direction is better.

3.5 Baselines

- *Chosen baseline for token selection (all DAG compatible)*
- 4.0: *k random*

- 4.1: degree based: Most children, Highest degree, Highest degree with $D_j = 1$
- 4.2: centrality based: Closeness centrality, Page rank, Personalized page rank
- 4.3 ours, and also our ablated version: Discrete set cover

We compare our method against seven baselines from three families, plus one ablation of our own method, summarized in Table 4. All of them are chosen to be well defined on a DAG; *[NOTE: per your feedback, eigenvector centrality has been removed from this list entirely – its Perron–Frobenius uniqueness guarantee requires strong connectivity, which a DAG cannot have, so it is no longer part of this draft even as a caveated option. It remains a real baseline in the codebase/notebooks; this is a paper-scope decision, not a code change.]* Every baseline ranks the full candidate pool V_D once and is then truncated to the top- k prefix of that ranking, exactly as our own method’s selection history is truncated – so every method in Section 4 is compared at matched k .

The discrete set-cover ablation is the one entry that is not a baseline in the usual sense: it isolates what the soft, λ -decayed, multi-hop coverage of Eq. (1) buys over textbook hard-coverage greedy, using the *same* selection mechanism, so any difference in Section 4 can be attributed to the coverage formulation itself rather than to the greedy procedure around it.

3.6 Expert acceptability evaluation

- evaluation of acceptability of tokenized version versus original concept, by clinical expert.

The metrics above are all automatic and intrinsic to the graph; they do not ask whether a person would actually recognize the tokenized representation as faithful to the original concept. We close this gap with a clinical expert evaluation: for a sample of mapped concepts, the expert is shown the original concept alongside its context-tree tokenization (Section 3.3) and rates whether the tokenized version still conveys an acceptable representation of the original concept’s meaning. *[NOTE: per your feedback, this is now firm in-paper content (no longer “(maybe)”), and it is intrinsic quality evaluation, distinct from the downstream/assistive-clinical-decision-task validation named in Section 6 – the two are not the same thing and this draft keeps them separate.]* **[DOUBT: still need from you: how many concepts/experts, what rating scale (binary acceptable/not, Likert, free-text justification?), and whether inter-rater agreement is reported if there is more than one expert – these design choices affect how this subsection should be written up and what Section 4 needs to report alongside the automatic metrics.]**

4 Results

4.1 Data

- *use all concepts SNOMED used and mapped by our semantician in HUGData*

We use every SNOMED CT concept mapped by our institution’s semantician team in HUGData as the mapped set M . *[NOTE: "HUGData" is used here exactly as it appears in your own `config.py` path names – flag if this should instead be spelled out or anonymized for the paper.]*

4.2 Candidate pool

- *how to get subgraph and narrow down the candidate pool, then give the number 400k -> 70k*

Before comparing methods, we first need to know how large the candidate pool actually is. Candidate tokens are restricted to the D -hop neighborhood of M (Section 3.1), which narrows the pool from $|V| \approx 400,000$ SNOMED concepts to **75,964** concepts under the current setting ($D = 9$), spread across **2** weakly connected components (Table 2). *[NOTE: also noted in Section 2: this is barely larger than the old $D=3$ figure ($\sim 75,634$) despite the much larger depth budget – the ego-graph around M saturates quickly.]* All methods compared below select their tokens from this same 75,964-concept pool, so any difference between them is a difference in *which* tokens are chosen, not in what they had access to.

4.3 General trend across k

- *General trend: k VS all metrics evolution*

With the candidate pool fixed, we now vary the token budget k and ask how quickly each method reaches good coverage. Table 5 shows semantic coverage $F_D(T)$ (evaluated at $\lambda = 1$, Section 3.4) against k , for one representative member of each family (Section 3.5): *highest degree* (degree-based), *personalized PageRank* (centrality-based), our own selector at $\lambda = 1$ and, separately, the full λ sweep, the *discrete set-cover* ablation, and *k-random*.

Three findings hold across the entire tested range ($k \in [500, 19,500]$, i.e. up to roughly half of $|M| \approx 40,000$), not just at the k -values tabulated above. First, every *structured* method – ours, both degree-based, and all three centrality-based baselines – reaches semantic coverage ≥ 0.999 somewhere in the tested range, while *k-random* does not: it only reaches 0.804 at the largest k tested, still well short of every structured method. Second, our own method’s envelope (best over $\lambda \in \{0.2, \dots, 1.0\}$) is the fastest of all five families to reach ≥ 0.999 coverage, doing so already at $k = 500$, the smallest budget tested – ahead of centrality ($k = 1,000$), degree ($k = 3,000$), the discrete-set-cover ablation (never reaches 0.999 within the tested range, topping out at exactly 0.999 only at

$k = 19,500$), and random (never reaches it). Third, k -random’s gap to every structured method does not close within the tested range: it is still roughly 0.2 below the nearest structured method even at $k = 19,500$.

4.4 Comparison across baseline families

- *analysis of metrics among different families of candidate selection, and the characteristics:*
- *degree based: may select semantic overlapped concept, less semantic preservation, redundant selection in same branch(?)*
- *centrality based: random walk based, less semantic preservation (?), overlapped semantics(?), maybe less efficiency (?)*
- *ours*
- *in this part, for each family, i will plot the upper envelop curve of metric performance*

The overall speed ranking above raises a follow-up question: does it hold *within* each family too, and does it hold on every metric, not just semantic coverage? We check each family in turn, then compare all of them against ours directly.

Degree-based. Within this family, *highest degree* (all relation types) dominates *most children* (IS_A only) on distance score, conciseness, uniqueness entropy, and unique rate at every k we sampled. Semantic coverage is the one exception, and it is narrow enough to deserve its own table rather than a parenthetical number: Table 6 shows that between $k = 2,500$ and $k = 4,500$, *most children* briefly has *higher* semantic coverage than *highest degree*, before *highest degree* overtakes it again at $k = 5,000$ and stays ahead for the rest of the tested range.

[NOTE: this local crossover is real, not a plotting artifact – verified directly against the parquet rows – but it is narrow enough that the headline claim should be “highest degree dominates most children almost everywhere,” not “always.” The pattern you predicted qualitatively (non-hierarchy relations carrying extra coverage signal that a pure-IS_A heuristic misses) does hold as the overall story – just not exceptionlessly.]

Centrality-based. *Personalized PageRank*, the one M -aware member of this family, reaches near-ceiling coverage fastest within the family ($k = 1,000$), ahead of plain PageRank and closeness centrality – consistent with the idea that restarting on M rather than uniformly should help when the objective is itself about covering M .

Ours vs. every baseline family, on coverage speed. At small k (500–1,000, i.e. roughly 1–2.5% of $|M|$), our λ -sweep envelope is the only family already at or above 0.998 semantic coverage; every baseline family needs a larger k to catch up (Table 5).

Discriminative power (uniqueness entropy / unique rate). Coverage speed is not the whole story, so Table 7 checks the two discriminative-power metrics directly, at a small k and a large one. **[DOUBT: your hypothesis**

that degree-based methods would show *lower* discriminative power than ours, from selecting semantically-overlapping candidates, is only partly borne out.]

At $k = 1,000$, the best degree-based member is close to our envelope on unique rate (0.588 vs. 0.594) – not the large gap the hypothesis predicted. At $k = 15,000$, a real gap does appear, but in the other direction: centrality-based baselines slightly *overtake* our envelope on both metrics. This is the one place in this sweep where a baseline family is not dominated by ours, and it is worth stating plainly: "ours wins on every metric at every k " would not be an accurate claim, even though it wins the coverage-speed race described above.

Ablation: soft vs. hard coverage. Finally, we check what the soft coverage formulation itself is buying, independent of the greedy mechanism around it. The discrete set-cover baseline (Section 3.5) uses the same lazy/CELF greedy mechanism and approximation guarantee as our method, but a binary rather than λ -decayed coverage signal. It is the slowest of all *structured* methods to approach ceiling coverage: it never reaches ≥ 0.999 within the tested range (Table 5), behind every degree-based and centrality-based baseline as well as our own method. This is a direct empirical confirmation that the soft, distance-aware formulation of Eq. (1) buys something concrete over classical hard-coverage greedy, not just a theoretical nicety.

4.5 Effect of the distance penalty λ

- *analysis of metrics between our methods, with respect to impact of lamda*
- *extreme case: lam = 0.0: force to choose the mapped concepts.*
- *trade off lambda vs diff metrics.*
- *token reuse rate (need to be implemented)*
- *point out: best config depends on personal choice.*

The comparisons above all use a single representative λ per family. We now ask the question specific to our method: what does λ itself trade off? Table 8 shows five metrics at one representative k ; the prose below states which patterns hold at that single k and which hold across the full sweep.

Checking every one of the 39 tested k values, not just the $k = 10,000$ snapshot in Table 8, three distinct patterns emerge:

Monotonically improving toward $\lambda = 1$ (100% of k values). Semantic coverage and unk rate improve monotonically as λ increases from 0.2 to 1.0, at every single tested k , always best at $\lambda = 1.0$. Uncovered rate follows the same direction but saturates to exactly 0 well before $\lambda = 1.0$: by $\lambda \geq 0.6$ it is already 0 at every $k \geq 1,500$, and even $\lambda = 0.2$ reaches 0 by $k = 5,000$.

Peaked at $\lambda \approx 0.8$ (large majority of k values). In the other direction, distance score is maximized at $\lambda = 0.8$ at *all* 39 tested k values, dropping again at $\lambda = 1.0$ every time (e.g. $0.863 \rightarrow 0.728$ at $k = 10,000$). The same peak shows up, slightly less consistently, in three other metrics: tree complexity

is minimized at $\lambda = 0.8$ at 38 of 39 k values, conciseness at 34 of 39, and uniqueness entropy at 30 of 39 ($\lambda = 0.6$ is the usual runner-up). This confirms your outline’s prediction that these metrics trade off against raw coverage – and shows that $\lambda = 0.8$ is consistently, not just occasionally, the best single setting for this group.

Its own pattern: exact rate. Exact rate follows neither group above. Among the positive λ values, it is highest at $\lambda = 0.2$ at every tested k , decreasing overall as λ increases toward 1 – moving *opposite* to semantic coverage and unk rate. The decrease is not perfectly monotonic (at $k = 10,000$, $\lambda = 1.0$ ’s 0.185 partially rebounds above $\lambda = 0.8$ ’s 0.155), but $\lambda = 0.2$ is, without exception, the best positive- λ choice for this metric.

The degenerate case $\lambda = 0$. Setting $\lambda = 0$ is not merely a bad setting; Eq. (1) shows it is a qualitatively different objective. At $\lambda = 0$, every “other-wise” term vanishes, so $S_h(u, T) = S_0(u, T) = \mathbf{1}[u \in T]$ at every horizon, and $F_D(T)|_{\lambda=0}$ reduces exactly to $\text{exact_rate}(M, T)$: selecting at $\lambda = 0$ optimizes exact-match rate, not coverage. The sweep confirms this is not just an algebraic curiosity. Across all 39 k values, the $\lambda = 0$ selection has *strictly the highest* exact rate of every tested λ at every single k (e.g. 0.217 at $k = 10,000$, vs. 0.199 for the next best, $\lambda = 0.2$) – exactly as predicted, since this is directly what $\lambda = 0$ optimizes – and, at the same time, *strictly the lowest* semantic coverage of every λ at every single k , dramatically so at small k : 0.012 at $k = 500$, actually below k -random’s 0.050 at the same budget, and still only 0.491 at the largest tested $k = 19,500$ (vs. ≥ 0.999 for every $\lambda \geq 0.2$). *[NOTE: note for precision: the selection-time objective at $\lambda = 0$ literally equals exact_rate, but the reported semantic_coverage column is always evaluated at $\lambda = 1$ (Section 3.4’s convention), so the two numbers are not the same quantity computed twice – the point is that a token set optimized for exact_rate performs catastrophically under the standard $\lambda = 1$ coverage evaluation everyone else is judged by.]*

Token reuse rate. **[DOUBT: confirmed not yet implemented in eval.py – this subsection stays a placeholder until that metric exists in code; happy to help define/implement it once you specify what “reuse” should mean here (e.g. how many distinct mapped concepts a single token ends up representing, across the whole context-tree population).]**

Practical takeaway. Putting the four patterns together, $\lambda \approx 0.8$ is a reasonable default when concise, nearby representations matter as much as raw coverage: it costs essentially nothing in semantic coverage or unk rate relative to $\lambda = 1$ (both already at or near ceiling by moderate k), while giving the best distance score, tree complexity, and usually conciseness and discriminative power in the whole sweep. $\lambda = 1$ remains the right choice if raw coverage at the smallest possible k is the only thing that matters, or if a downstream use case specifically wants the highest possible exact-match rate among large- λ settings. **[DOUBT: “depends on personal choice” – happy to sharpen this into an explicit recommendation/decision rule if you tell me what the downstream FM actually optimizes for (e.g. embedding-table size vs. per-concept fidelity).]**

5 Discussion

- *Empty for the moment*

[DOUBT: Nothing drafted as prose here yet, per your note. Section 4 now has real numbers, so this section can close the loop back to Section 1’s motivation whenever you’re ready to draft it. Suggested threads to pick up, in a plausible reading order:]

- How much smaller a vocabulary does structured (greedy / degree / centrality) selection buy over the “keep everything” or “random” baselines from the Introduction’s gap paragraph, at what cost in semantic coverage or distance score – i.e. turn Section 4.3/4.4 into a direct answer to “does this solve the FM vocabulary-size problem better than the alternatives listed in the Introduction.”
- What the $\lambda = 0$ result (Section 4.5) implies practically: since λ trades off raw coverage against proximity/conciseness, is there a principled way to pick λ for a given downstream FM, or is it genuinely a design knob left to the practitioner (tying back to your Results outline’s closing point)?
- Revisit the controllability/interpretability claim from Section 1’s contribution list, and the expert acceptability results from Section 3.6: what does a reader/expert actually get to inspect or judge that a codebook-based tokenizer would not let them inspect? A short worked example (reusing the context-tree figure from Section 3.3) would make this concrete rather than asserted.

6 Limitations

The results above show what the method achieves on the metrics we defined. Four limitations bound how far those results should be read.

6.1 Graph structure constraints

- *Graph structure constraints – incomplete semantic definition, which our tokenizer can not do shit*

The context tree and the coverage score (Section 3) can only ever reflect the relationships that are actually encoded in the graph. Where SNOMED CT’s own definition of a concept is incomplete – a relevant attribute simply not modeled as an outgoing edge – the tokenizer cannot compensate for this: an under-specified concept will look under-specified in its context tree regardless of which token set T is chosen. [DOUBT: I read “our tokenizer can not do shit” as “our method has no way to fix or detect this,” which is what the sentence above says – flag if you meant something more specific, e.g. a particular class of missing relationships you’ve already run into.]

6.2 Transductive method

- *transductive method for token selection*

A second limitation concerns how the method generalizes beyond the mapped set it was run on. Token selection (Section 3.2) is transductive: it optimizes coverage for a fixed, known M , and a token set selected for one institution’s mapped concepts is not guaranteed to transfer to a different mapped set without re-running selection.

6.3 No external validation on a clinical task

- *No external validation on clinical task*

A third, related limitation concerns what “quality” was actually validated against. This paper evaluates tokenization quality intrinsically (the metrics of Section 3.4) and, directly, via clinical-expert acceptability ratings of the tokenized representation against the original concept (Section 3.6). What it does not do is validate the tokenization on an actual downstream, assistive clinical decision task – i.e. show that a model trained on this reduced vocabulary performs a real clinical decision-support function (diagnosis support, risk prediction, etc.) at least as well as one trained on an alternative vocabulary. *[NOTE: per your feedback, this is a distinct claim from the expert acceptability evaluation in Method – that one is about representation fidelity judged by an expert, this one is about end-task clinical utility, and only the first is in scope for this paper.]*

6.4 Requires a DAG

- *Not working on Non-DAG*

Finally, the method’s applicability is bounded by the same structural assumption made throughout the paper. The tokenizer’s core mechanism – IS_A transparency, non-IS_A subcontexts, and the coverage recursion’s reliance on well-defined finite hop distances – assumes an acyclic graph. It does not extend as-is to a vocabulary with cycles, or to a general knowledge graph where the child/parent asymmetry of Section 2 does not hold.

7 Future Work

The limitations above point to four directions for extending this work.

7.1 Inter-ontology tokenization

- *if mapping inter ontology snomed and others, can show case that we can tokenize concepts in 1 ontology with other concept tokens in other ontology*

Where a mapping between SNOMED CT and another ontology exists, the same selection and tokenization mechanism could be applied across ontologies: tokens selected in one vocabulary could represent concepts from another, extending the interoperability argument from Section 1 to the token level itself. *[NOTE: this is not claimed as a contribution of the current paper (see Section 1) – it lives here only, as you asked.]*

7.2 Redundancy and token swaps

[NOTE: pulled from your PDF’s §8 “Perspectives” – not in main.tex’s outline, but already sketched there.]

A second direction addresses a property of the greedy procedure itself rather than of the ontology. The greedy algorithm of Section 3.2 never removes a token once selected. A token can become nearly redundant once later tokens end up covering the same concepts it was originally selected for; a marginal-*loss*-based pass could identify and swap out such tokens after the fact.

7.3 Primitive vs. fully-defined concepts

[NOTE: also pulled from your PDF’s §8.]

A third direction questions an assumption made throughout Section 3: that expanding a concept is lossless. Expanding a concept via its outgoing relationships (Section 3.3) implicitly assumes that doing so is lossless. For primitive SNOMED CT concepts – those not fully defined by their stated relationships – this may not hold. A reconstructibility factor $\alpha(u) \in [0, 1]$ ($\alpha(u) = 1$ for fully defined concepts, $\alpha(u) = \eta < 1$ for primitive ones) could discount the propagated coverage of Eq. (1) accordingly, while still treating a primitive concept selected *as its own token* as fully covered (coverage 1), since it is then preserved atomically rather than reconstructed from its relationships.

7.4 Combining with an ontology-similarity measure

[NOTE: also pulled from your PDF’s §8, which already cites a reference for this ([6] in your PDF).]

A fourth, more speculative direction would change the objective itself rather than build on top of it. An independently developed ontology-based similarity measure **[CITE: ontology-based similarity measure referenced in your PDF’s Section 8 / reference [6]]** could evaluate how well semantic neighborhoods are preserved and how severe token collisions are, and could potentially be folded into the marginal-gain computation itself, e.g. as a facility-location-style term – provided the submodularity and monotonicity properties the greedy guarantee depends on (Section 3.2) are re-verified for the modified objective.

Metric	Family	Better	Definition
Conciseness	Efficiency	lower	Mean number of token leaves per concept’s context tree.
Tree complexity	Efficiency	lower	Mean number of contexts (root plus subcontexts) per tree.
Distance score	Fidelity	higher	$1 - \bar{d}/(D+1)$, where $\bar{d}$ is the mean hop-distance at which tokens are found (uncovered branches counted at $D+1$). 1.0 means every branch found a token immediately.
Uniqueness entropy	Discriminative power	higher	Normalized Shannon entropy of context-tree signatures across M ; low when many concepts collapse onto the same tree shape.
Unique rate	Discriminative power	higher	Per-concept decomposition of the above: fraction of concepts whose signature is not shared by any other concept.
Exact rate	Discriminative power	higher	Fraction of mapped concepts that are themselves selected as a token (a 0-hop, exact representation); maximally discriminable by construction.
Semantic coverage	Semantic content	higher	$F_D(T)$ itself, always reported at $\lambda = 1$ regardless of the λ used at <i>selection</i> time (see [DOUBT: confirm this evaluation convention]).
Uncovered rate	Semantic content	lower	Fraction of concepts with <i>zero</i> tokens anywhere in their tree.
Unk rate	Semantic content	lower	Fraction of concepts with <i>at least one</i> uncovered branch; by construction, uncovered rate $\leq$ unk rate.

Table 3: The eight evaluation metrics, grouped by family, with direction of improvement and definition.

Baseline	Family	Rule	DAG note
k -random	Random	Independent uniform sample of k candidates without replacement, repeated over seeds and averaged (no single ranking to truncate).	Trivial
Highest degree	Degree-based	Number of distinct predecessors within D hops, over every relationship type.	Compatible
Highest degree, $\Delta \leq 1$	Degree-based	Same score, restricted to direct (1-hop) predecessors.	Compatible
Most children	Degree-based	Same D -hop predecessor count, restricted to the IS_A-only subgraph.	Compatible
PageRank	Centrality	Stationary distribution of a random walk with restart probability $1 - \alpha$, $\alpha = 0.85$ [CITE: Brin & Page, PageRank].	Compatible
Personalized PageRank	Centrality	Same recursion, restarting uniformly on M instead of on all of V – the one M -aware centrality baseline [CITE: personalized / topic-sensitive PageRank, e.g. Haveliwala].	Compatible
Closeness centrality	Centrality	Reciprocal of the mean shortest-path distance <i>to</i> a candidate from every node that can reach it (in-degree-oriented, Wasserman–Faust improved) [CITE: Freeman, closeness centrality].	Compatible
Discrete set-cover (ablation)	Ours, ablated	Same lazy/CELF greedy mechanism and $(1-1/e)$ guarantee (Eq. (2)) as our method, but $S_h(u, T)$ replaced by a binary "within D hops of some token" indicator – classical maximum-coverage greedy.	Compatible

Table 4: Baselines compared against our method, grouped by family. Eigenvector centrality is excluded (see note below).

k	500	1,000	5,000	10,000	19,500
Highest degree	0.981	0.985	1.000	1.000	1.000
Personalized PageRank	0.998	0.999	1.000	1.000	1.000
Ours, $\lambda = 1.0$	1.000	1.000	1.000	1.000	1.000
Ours, $\lambda = 0.8$	0.998	1.000	1.000	1.000	1.000
Discrete set-cover (ablation)	0.977	0.978	0.991	0.995	0.999
k -random	0.050	0.073	0.351	0.587	0.804

Table 5: Semantic coverage $F_D(T)$ (evaluated at $\lambda = 1$) vs. token budget k , one representative per family. Full λ sweep in Table 8.

k	2,500	3,000	3,500	4,000	4,500	5,000
Highest degree	0.9962	0.9963	0.9963	0.9981	0.9988	0.9999
Most children	0.9978	0.9990	0.9995	0.9996	0.9998	0.9998

Table 6: Semantic coverage, highest degree vs. most children, around the crossover region. Bold marks the higher value in each column.

k	Best member of family	Uniqueness entropy	Unique rate
1,000	Degree-based (highest degree, $\Delta \leq 1$)	0.920	0.588
	Centrality-based (personalized PageRank)	0.928	0.530
	Ours ($\lambda = 0.4$ envelope)	0.936	0.594
15,000	Degree-based (highest degree, $\Delta \leq 1$)	0.985	0.883
	Centrality-based (personalized PageRank)	0.989	0.903
	Ours ($\lambda = 0.8$ envelope)	0.987	0.893

Table 7: Uniqueness entropy and unique rate at $k = 1,000$ and $k = 15,000$, best member of each baseline family vs. our envelope. Bold marks the highest value in each block.

λ	0.0	0.2	0.4	0.6	0.8	1.0
Semantic coverage	0.245	0.996	0.999	1.000	1.000	1.000
Unk rate	0.782	0.131	0.054	0.021	0.012	0.000
Distance score	0.260	0.804	0.838	0.855	0.863	0.728
Conciseness	2.591	9.985	8.940	7.971	7.411	11.500
Exact rate	0.217	0.199	0.183	0.169	0.155	0.185

Table 8: Effect of λ on five metrics at $k = 10,000$ (representative of the pattern at every k we checked – see prose for how consistently each pattern holds across the full $k = 500$ –19,500 range).